

Contenido

HITO DE INTEGRACIÓN SQA

Portal de Consejeras

Curso

[nombre del curso]

Catedrático

[nombre]

Grupo

Brandon Filemón Arreaga · Juan Carlos
Vásquez · Juan Francisco Arriola

Fecha

[fecha]

Versión

1.0

Introducción

Este documento entrega los tres artefactos que pide el hito:

1. Matriz de Roles y Responsabilidades (RACI)
2. Especificación de Requisitos No Funcionales (NFR), según ISO/IEC 25010
3. Plan Inicial de Pruebas: estructura de pruebas unitarias e integración

Al final se responde explícitamente al checklist de auditoría de clase.

1. Matriz de Roles y Responsabilidades (RACI)

1.1 Qué es una matriz RACI

Para cada actividad del proyecto, define quién:

Letra	Significa	En corto
R	Responsable	Quien hace el trabajo
A	Aprueba	Quien responde por el resultado final
C	Consultado	A quien se le pide opinión antes de decidir
I	Informado	A quien se le avisa cuando ya está hecho

Regla de la técnica: cada actividad debe tener exactamente una **A**. Si nadie responde por algo, o si dos personas creen que son las responsables finales, la matriz está mal hecha.

1.2 Los cuatro roles en nuestro equipo

Somos tres integrantes cubriendo cuatro roles, igual que ya lo documentamos en el Hito 1 (sección 2.7). Aquí se formaliza cuál rol corresponde a cuál actividad.

Rol	Qué significa	Quién lo asume
PO (Product Owner)	Define qué debe hacer el sistema y qué se considera terminado	Brandon Arreaga
Dev (Desarrollador)	Escribe el código de la aplicación	Los tres, según la historia asignada
QA (Aseguramiento de Calidad)	Diseña y ejecuta las pruebas	Juan Carlos Vásquez
DevOps	Mantiene el pipeline y el despliegue	Brandon Arreaga

1.3 La matriz

Actividad	PO	Dev	QA	DevOps
Definir historias de usuario y reglas de negocio	A/R	C	C	I
Escribir el código de las pantallas y la lógica	C	A/R	I	I
Diseñar los casos de prueba (RTM)	I	C	A/R	I
Escribir y ejecutar pruebas unitarias	I	R	A	I
Escribir y ejecutar pruebas de integración	I	C	A/R	C
Configurar y mantener el pipeline (QA Gate)	I	C	C	A/R
Revisar y aprobar Pull Requests	I	R	A	C
Registrar y dar seguimiento a defectos (Kanban)	I	C	A/R	I
Publicar el sitio (GitHub Pages)	I	C	I	A/R
Documentación del proyecto	A	R	R	I
Defensa técnica en clase	A	R	R	R

Cómo leerlo: en “Revisar y aprobar Pull Requests”, QA es quien aprueba (A) y Dev es quien hace el cambio que se revisa (R) — esto refleja la regla que ya tenían escrita desde el Hito 1: nadie aprueba su propio código.

1.4 Por qué esta repartición y no otra

- El **PO** concentra las decisiones de alcance porque alguien tiene que responder cuándo algo “cuenta” como terminado; se lo dimos a quien administra el repositorio, porque ya tomaba esas decisiones de facto.
 - El **QA** aprueba el trabajo de Dev, nunca al revés — así se mantiene la separación que exige la sección 5.1 del Máster Test Plan.
 - El **DevOps** es responsable único del pipeline y el despliegue, porque son piezas técnicas que necesitan un dueño claro para no romperse por descuido de varios.
-

2. Especificación de Requisitos No Funcionales (NFR)

2.1 Por qué esto no es opcional

Un requisito funcional dice **qué** hace el sistema (“la consejera puede agregar un producto”). Un requisito no funcional dice **qué tan bien** lo hace, y tiene que ser medible con un número, no con un adjetivo.

Mal (adjetivo ambiguo)	Bien (medible)
El catálogo debe cargar rápido	El catálogo debe mostrarse en 2 segundos o menos
El sistema debe ser seguro	El 100% de las entradas maliciosas probadas deben ser rechazadas
La interfaz debe ser bonita	<i>(esto no es un NFR verificable; se excluye)</i>

Un NFR sin número no se puede probar, y si no se puede probar, no sirve para este curso.

2.2 Los seis NFR del proyecto

Clasificados según las características de calidad de **ISO/IEC 25010**.

NFR-01 • Eficiencia de desempeño

Requisito	El catálogo de productos debe mostrarse completo en 2 segundos o menos
Cómo se mide	Herramientas de desarrollador del navegador (pestaña <i>Network</i>), midiendo desde la carga de <code>catalogo.html</code> hasta que se pintan los 8 productos
Umbral de aceptación	≤ 2000 ms, en una conexión de banda ancha normal

NFR-02 • Usabilidad

Requisito	Una consejera que usa el sistema por primera vez debe poder completar el flujo completo
------------------	---

	(iniciar sesión – agregar un producto – enviar el pedido) en 5 minutos o menos, sin ayuda externa
Cómo se mide	Prueba con un usuario que no participó en el desarrollo, cronometrado desde que abre <code>login.html</code> hasta que ve el número de confirmación
Umbral de aceptación	≤ 5 minutos, con un máximo de 1 error de manejo
NFR-03 • Fiabilidad	
Requisito	Ningún pedido debe perderse ni duplicarse al enviarse, incluso ante un doble clic accidental
Cómo se mide	Prueba de integración INT-03 (ver sección 3) y caso manual CP-18 de la matriz de trazabilidad
Umbral de aceptación	0 pedidos perdidos y 0 duplicados en 20 intentos consecutivos
NFR-04 • Seguridad (dentro del alcance sin servidor)	
Requisito	Todo campo de texto debe rechazar o neutralizar intentos de inyección de código (XSS) sin que se ejecute ningún script
Cómo se mide	Ingresa en los campos de búsqueda y cantidad textos como <code><script>alert(1)</script></code> y verificar que se muestren como texto plano, nunca ejecutados
Umbral de aceptación	100% de los intentos probados quedan neutralizados
Limitación declarada	No cubre autenticación real, porque el login es simulado (ver Documento SQA, sección 3.4)
NFR-05 • Mantenibilidad	
Requisito	Ninguna función del código debe superar una complejidad ciclomática de 10 ni 50 líneas
Cómo se mide	Automáticamente, con las reglas <code>complexity</code> y <code>max-lines-per-function</code> de ESLint, en cada ejecución del QA Gate
Umbral de aceptación	0 funciones que excedan el límite; si una lo excede, el QA Gate falla y bloquea la integración

NFR-06 • Compatibilidad

Requisito

El sitio debe verse y funcionar sin errores visuales en las versiones actuales de Chrome, Firefox y Edge, en pantalla de escritorio (1920×1080) y de celular (390×844)

Cómo se mide

Inspección manual abriendo el sitio publicado en cada navegador y cada resolución, siguiendo la sección 5.9 del Máster Test Plan

Umbral de aceptación

0 errores visuales o funcionales en las 6 combinaciones (3 navegadores × 2 resoluciones)

2.3 Resumen de trazabilidad

NFR	Característica ISO 25010	Verificado por
NFR-01	Eficiencia de desempeño	Medición manual con DevTools
NFR-02	Usabilidad	Prueba con usuario externo
NFR-03	Fiabilidad	INT-03, CP-18
NFR-04	Seguridad	Prueba manual con entradas maliciosas
NFR-05	Mantenibilidad	ESLint, automático en el QA Gate
NFR-06	Compatibilidad	Matriz de compatibilidad (Test Plan 5.9)

3. Plan Inicial de Pruebas: Unitarias e Integración

3.1 Qué cambia respecto al Hito 1

En el Hito 1 ya teníamos **pruebas unitarias**: prueban una función sola, aislada de las demás. Este hito agrega **pruebas de integración**: prueban que varias funciones de distintos archivos trabajan bien en conjunto, siguiendo un flujo completo.

Tipo	Qué prueba	Cuántas	Dónde
Unitarias	Una función aislada	12	tests/ pedido.test.js
Integración	Varios módulos combinados	5	tests/ integracion/ flujo-

Tipo	Qué prueba	Cuántas	Dónde
Total automatizado		17	<code>pedido.test.js</code> Corren juntas con <code>npm test</code>

3.2 Componentes de alto riesgo que se cubren

Se priorizaron según el registro de riesgos del Máster Test Plan (sección 4).

Riesgo	Descripción	Prueba de integración que lo mitiga
R-01	Se envía un pedido sin llegar al mínimo	INT-01, INT-05
R-03	El pedido se duplica al enviarse dos veces	INT-03
R-04	Se accede sin sesión activa	INT-04
RN-03 (regla)	Se agrega un producto agotado	INT-02

3.3 Las cinco pruebas de integración

ID	Qué hace	Módulos que combina
INT-01	Arma un pedido válido de principio a fin: valida disponibilidad, valida cantidad, calcula el total, confirma que puede enviarse y lo registra en el historial	<code>pedido.js + sesion.js</code>
INT-02	Verifica que un producto agotado nunca llega a sumarse al total del pedido	<code>pedido.js</code>
INT-03	Comprueba que, tras un envío exitoso, el pedido vacío ya no puede reenviarse y el historial queda con exactamente un registro	<code>pedido.js + sesion.js</code>
INT-04	Confirma que un pedido válido en sus montos igual se bloquea si no hay sesión activa	<code>pedido.js</code>
INT-05	Verifica que el total guardado en el historial coincide exactamente con el calculado, incluso con decimales	<code>pedido.js + sesion.js</code>

3.4 Por qué estas cinco y no otras

Se eligieron las combinaciones donde un error en la **interacción** entre módulos sería más grave que un error en una función aislada. Por ejemplo, INT-03 no prueba si `calcularTotal()` funciona bien —eso ya lo prueba una unitaria—, prueba si el **flujo completo** de enviar y luego intentar reenviar se comporta como debe. Ahí es donde aparecen los defectos que las pruebas unitarias no pueden ver, como el defecto real que encontramos en el Hito 3 (los pedidos no se conservaban al reiniciar).

3.5 Cómo se ejecutan

Igual que las unitarias, forman parte del mismo comando:

```
npm test
```

Y corren automáticamente en el QA Gate en cada Pull Request, sin configuración adicional — el pipeline ya estaba preparado para descubrir cualquier archivo `*.test.js` en cualquier carpeta.

4. Respuesta al checklist de auditoría de clase

✓ Definición inequívoca de criterios de aceptación

Los 5 criterios de aceptación BDD del Hito 3 (`tests/aceptacion/pedido.feature`) ya están escritos en formato Dado/Cuando/Entonces, con valores concretos (Q400.00, Q399.99, cantidad 100), no con descripciones vagas.

✓ Eliminación de adjetivos ambiguos

La sección 2 de este documento reemplaza cualquier lenguaje de “rápido/bonito/seguro” por umbrales medibles en segundos, porcentajes y conteos.

✓ Configuración del pipeline de CI/CD para pruebas automáticas

Ya configurado desde el Hito 1 en `.github/workflows/qa-gate.yml`, verificado y corregido en la práctica: la primera ejecución falló por una incompatibilidad de versión de Node, se corrigió, y desde entonces corre en verde. Con este hito, el pipeline ahora ejecuta 17 pruebas automáticas en vez de 12.

Documento del Hito de Integración SQA. Los datos de negocio utilizados son ficticios y con fines académicos.